AI Engineer World's Fair

FROZEN ENCODERS · PROGRAM SEARCH · INFERENCE

Autoresearch
Test-Time Compute for Information Retrieval

SCAN FOR THE SLIDES

hanxiao.io/ttc-aie-sf

Han Xiao
VP of AI, Elastic

@hxiao · in/hxiao87 · arXiv:2605.11374

TEST-TIME COMPUTE

Spend more compute at inference, get a better answer.

Trade inference compute for quality: rather than a bigger model, the model does more work per query, along a smooth cost-versus-quality curve.

BEST-OF-N	sample many candidates, keep the best.
SELF-CONSISTENCY	majority-vote over sampled chains of thought.
VERIFIER SEARCH	spend inference FLOPs, climb a Pareto curve.

> Having a bot think for just **20 seconds** in a hand of poker got the same boosting performance as scaling up the model by **100,000×** and training it for 100,000 times longer.
>
> Noam Brown, OpenAI · TED AI, San Francisco

THE REFRAME

Search is test-time compute.

You wire trained embeddings, rerankers, multi-vector retrievers, and query expanders into a **pipeline at inference** to squeeze out relevance. You do not need a bigger model. You assemble more search at test time.

DON'T SCALE IT

one keyword match. Cheap, and probably not good enough.

SCALE IT

embed, retrieve, rerank, expand, fuse. More inference can buy more relevance.

The question is not whether the model is big enough. It is how much search pipeline you assemble at test time, and whether it pays off.

TWO INTERPRETATIONS

Two ways to manufacture that pipeline at test time.

A RECOMBINE THE GEOMETRY

An agentic loop writes **programs** over one frozen encoder: chunk, z-score, fuse channels, feed back. The pipeline is multi-pass embedding algebra.

the paper · this talk's core

B COMPOSE THE TOOLS

A small agent LLM **wires retrieval tools** (grep, embed, rerank, select_diverse) over a corpus under a budget. The pipeline is an agentic tool chain.

searchbox · later

Same move at two altitudes:
assemble structure at inference, not a bigger model.

INTERPRETATION A

"Small models gain nothing." But they are distilled from LLMs.

Test-time compute is said to belong to large reasoning models, with small models having nothing to gain. Yet today's small embedders are distilled or adapted from LLM backbones:

Mistral-7B →	E5-Mistral · SFR-Embedding · GritLM · NV-Embed · bge-en-icl
Qwen3 →	qwen3-embed · jina-embeddings-v5
Gemma 3 →	EmbeddingGemma
LLaMA →	RepLLaMA · LLM2Vec

The lineage from a large language model to a small deployable encoder is now the dominant recipe, not the exception.

If test-time compute lives in the LLM representation space, these distilled encoders should inherit it. Do they?

THE INTUITION

Scoring runs from one cosine to late interaction.

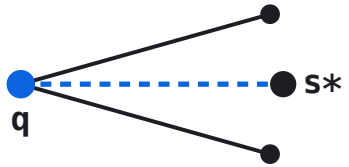
SINGLE-VECTOR COSINE

$\cos(q, d)$

one vector per document

FROZEN BASELINE

SENTENCE MAXSIM

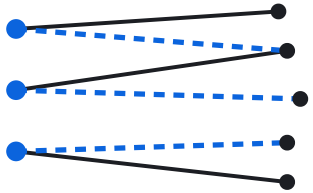


$\max_s \cos(q, s)$

same encoder, document split into sentence vectors

TEST-TIME STRUCTURE

COLBERT MULTI-VECTOR



$\sum_i \max_j \cos(q_i, d_j)$

every query token, max over all doc tokens

NEEDS A MULTI-VECTOR MODEL

Test-time compute adds structure over the same frozen vectors: chunk, feed back, fuse, and climb toward late interaction with no new model.

THE QUESTION

How much can a frozen single-vector encoder gain at inference alone?

NO RETRAINING | NO AUXILIARY MODEL | NO LEARNED PARAMETERS | ONE FROZEN ENCODER API

Prior test-time methods for retrieval each break at least one rule:

- **HYDE / QUERY2DOC** — an external LLM in the query path.
- **GQR** — a second, supervisory retriever.
- **METAEMBED** — trained extra parameters at deployment.

We forbid all three, and ask whether the gain scales with the compute spent.

AUTORESEARCH

Autoresearch: let the agent climb the hill.

An agent runs the research loop itself: change one file, run a short fixed-budget experiment, keep the change if the metric improved, otherwise revert. Repeat, overnight. It is hill-climbing, with an LLM as the mutation function.

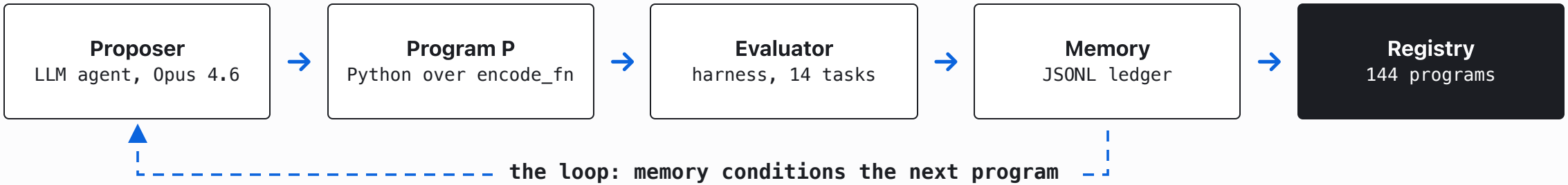
`change a file` → `run a short experiment` →

`keep if better, else revert` ↻

> **You're not touching any of the Python files like you normally would as a researcher. Instead, you are programming the `program.md` files that set up your autonomous research org.**
>
> Andrej Karpathy · AutoResearch, 2026

METHOD

An LLM agent writes programs over the frozen encoder, and a harness scores them.



$P : (\mathbf{Q}, \mathbf{D}, \mathbf{S}, \mathbf{ctx}) \to \mathbf{S}'$ an arbitrary deterministic Python function over the baseline cosine similarity. Each `encode_fn` call is one unit of test-time compute.

METHOD · 1 PROPOSER

The proposer is an LLM, used as the mutation function.

DESIGN

Opus 4.6 reads the current frontier source and the memory ledger, edits **one** Python program, and proposes the next. Keep it if the metric improves, else revert: hill-climbing, with the LLM as the mutation function. No human in the inner loop.

CAVEAT

It optimizes **exactly** the metric you hand it. Tell it to raise the in-domain mean and spend compute, and it will manufacture a 14.7× frontier that does not transfer. The objective is the steering wheel.

METHOD · 2 PROGRAM

The program is arbitrary Python over the frozen encoder.

DESIGN · THE INTERFACE

```
rerank(q_emb, d_emb, sim, *,
       encode_fn, q_texts, d_texts) -> scores
```

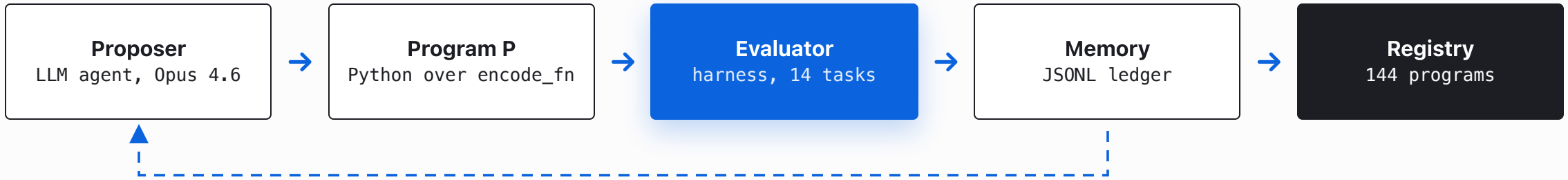
encode_fn is the test-time-compute budget. It re-encodes any text, switches the LoRA adapter (retrieval / passage / classification / matching), picks a Matryoshka dimension, or caps length. One call is one unit of compute.

CAVEAT · SIX TABOOS

Auto-rejected: hyperparameters, stochastic ops, task-specific routing, external models, learned parameters, trivial constant-only variants. The taboos force **universal structure**, not a per-task tuned config.

METHOD · 3 EVALUATOR

The harness scores every program on the same 14 tasks.



DESIGN

Every program runs on the same **14 MMTEB Tier-1 discovery tasks** (legal, financial, long-document, general). The harness scores `ΔnDCG@10` against the cosine baseline, plus a cost ratio (`encode_fn` calls per query). The same fixed budget every generation.

CAVEAT

The loop only ever sees these 14 tasks. The **19 held-out tasks never enter the loop**, so a program can win in-domain and still collapse out of domain. That gap is the entire experiment.

METHOD · 4 MEMORY

Every program is logged, and the log conditions the next one.

DESIGN

A JSONL ledger, one row per program: per-task `ΔnDCG@10`, win/tie/loss, cost ratio, parent, a novelty claim, and a post-hoc lesson. The proposer reads the frontier source plus this history, so each round builds on the last.

CAVEAT

Compounding cuts both ways: memory builds on real wins, but it also compounds whatever bias the objective has. A biased metric does not just mislead one program, it steers the whole lineage.

SETUP

What we search on, and what we test on.

j-v5-nano
239M · Jina
DISCOVERY MODEL

j-v5-small
568M · Jina, multi-LoRA
HELD-OUT

gemma-300m
303M · Gemma 3, no adapters
UNSEEN FAMILY

qwen3-0.6b
600M · Qwen3, no adapters
UNSEEN FAMILY

DISCOVERY

The loop runs on `j-v5-nano` over **14 MMTEB Tier-1 tasks** (legal, financial, long-document, general), corpora of 46 to 438 documents.

HELD-OUT

19 MMTEB Tier-2/3 tasks, none in discovery, corpora up to ~12,500 docs: summarization, legal and medical QA, fact verification, argument retrieval, temporal and commonsense reasoning. Touched once.

We score every (encoder, task) cell by ΔnDCG@10, the program's nDCG@10 minus the cosine baseline. Gemma and Qwen share no training data, tokenizer, or adapter design with the discovery model.

THE DISTINCTION

Cost is one number: how many extra times a program calls the encoder.

$$c = 1 + \frac{\text{extra encoder calls}}{\text{baseline calls}}$$

TEST-TIME STRUCTURE · C = 1

SoftCentroid: average the top-k document vectors you *already* computed, mix into the query, re-score.

```
q′ = q + mean(d_emb[top-k])
```

Zero extra encoder calls — only algebra on vectors you already have.

TEST-TIME COMPUTE · C > 1

FirstSent: take the top doc, *re-encode* its first sentence, mix into the query, re-score.

```
q′ = q + encode_fn(first sentence)
```

One extra encoder call per query — new text through the model.

Same move — enrich the query with document context. The only difference is the encoder call. Which one converts into quality that transfers?

TWO SEARCHES, SAME LOOP

We run the search under two admission rules.

COMPUTE-SPENDING

Admit a program if its in-domain mean ΔnDCG@10 beats every prior one. The proposer is encouraged to spend more inference.

→ a frontier from $c = 1.2$ to 14.7

TRANSFER-SELECTING

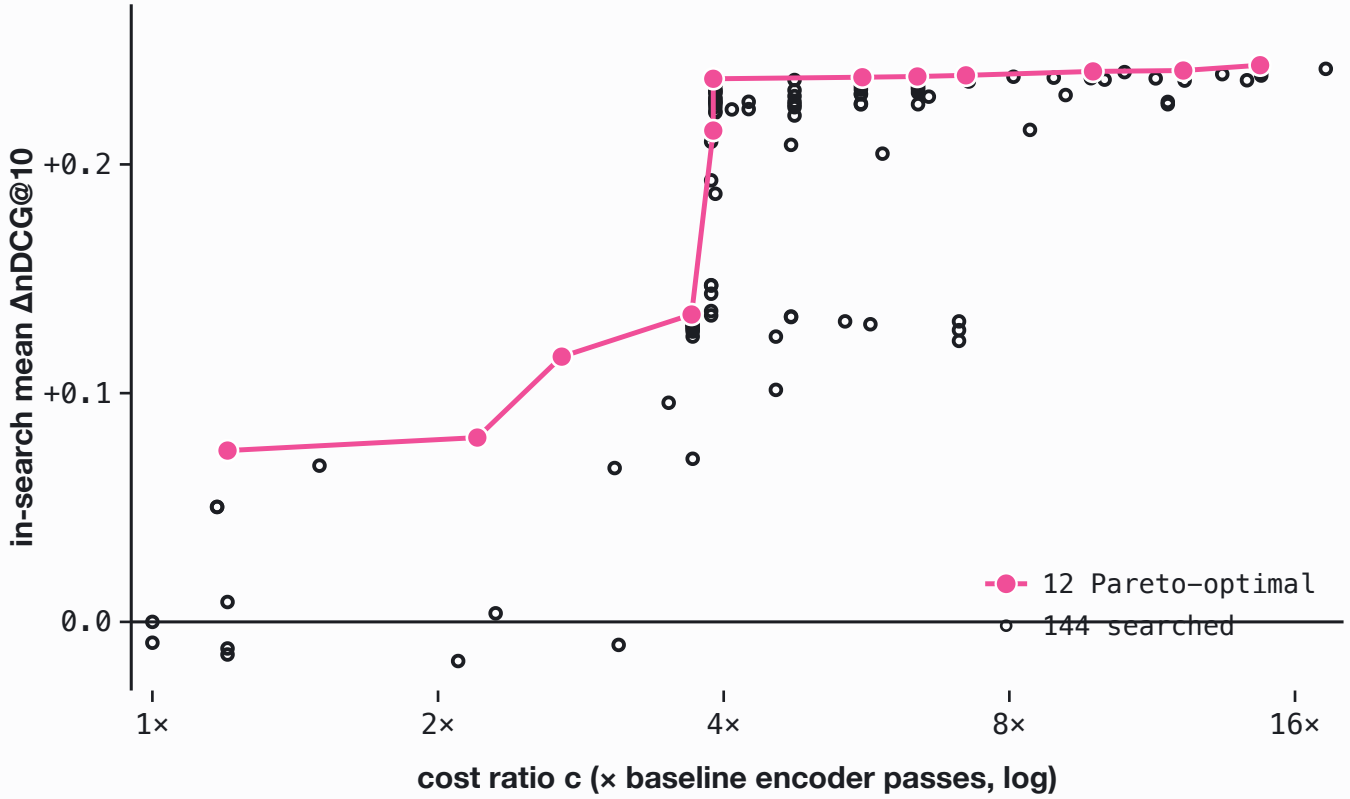
Admit only if a held-out validation split improves with no task regressing past 0.05. No reward for spending compute.

→ converges near $c = 1$

Neither search ever sees the 19 held-out evaluation tasks or the unseen encoder families.

WHAT COMPUTE BUYS, IN SEARCH

Told to spend compute, the search draws a clean Pareto curve.



144

programs searched over 144 generations

12

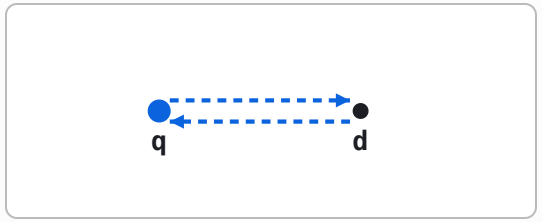
Pareto-optimal, from $c = 1.2$ to 14.7

In-search mean ΔnDCG@10 climbs from +0.07 to +0.24. This looks exactly like test-time-compute scaling.

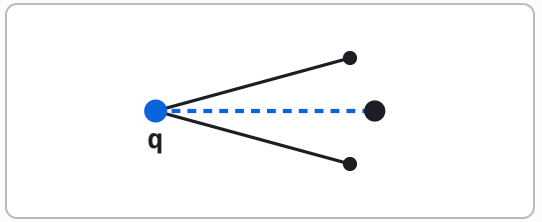
In-search only. The held-out test comes next.

WHAT THE SEARCH WROTE

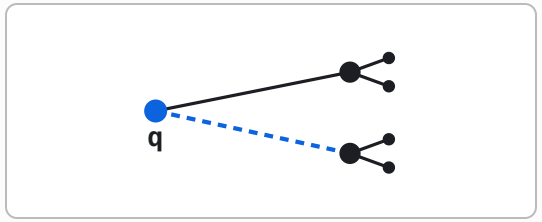
Twelve programs on the frontier, one frozen encoder.



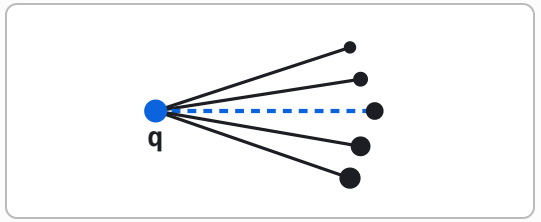
BidirZScore c=1.2



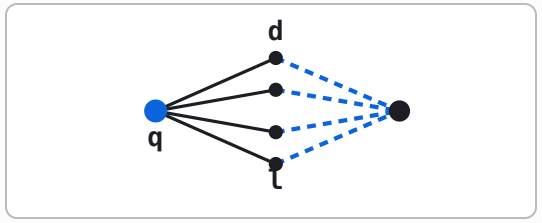
SentMaxSim c=2.2



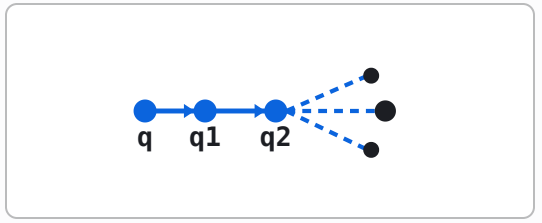
AdaptGranularity c=2.7



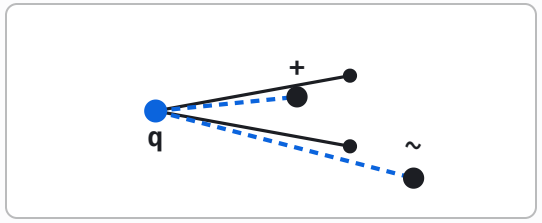
CoverageTriple c=3.7



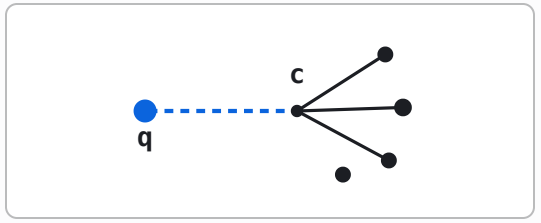
LexicalHybridRRF c=3.9



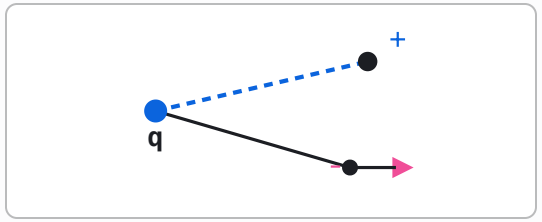
CrossRoundRRF c=3.9



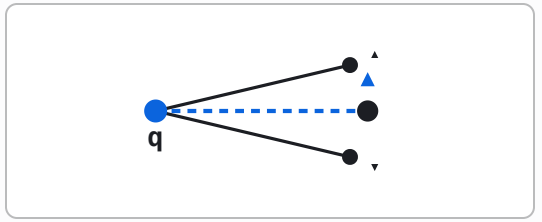
DiverseDualCtx c=5.6



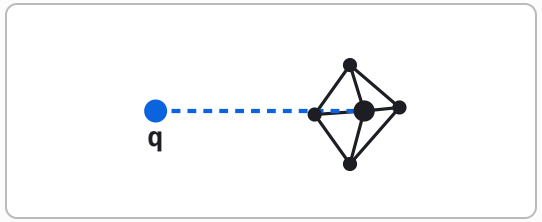
ConsensusRocchio c=6.4



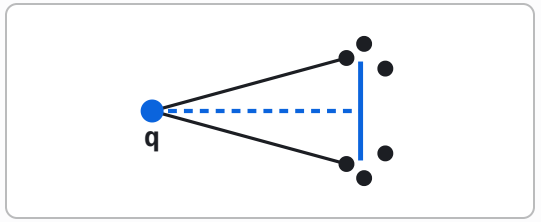
NegContrastive c=7.2



MomentumProg c=9.8



GraphCentrality c=12.2

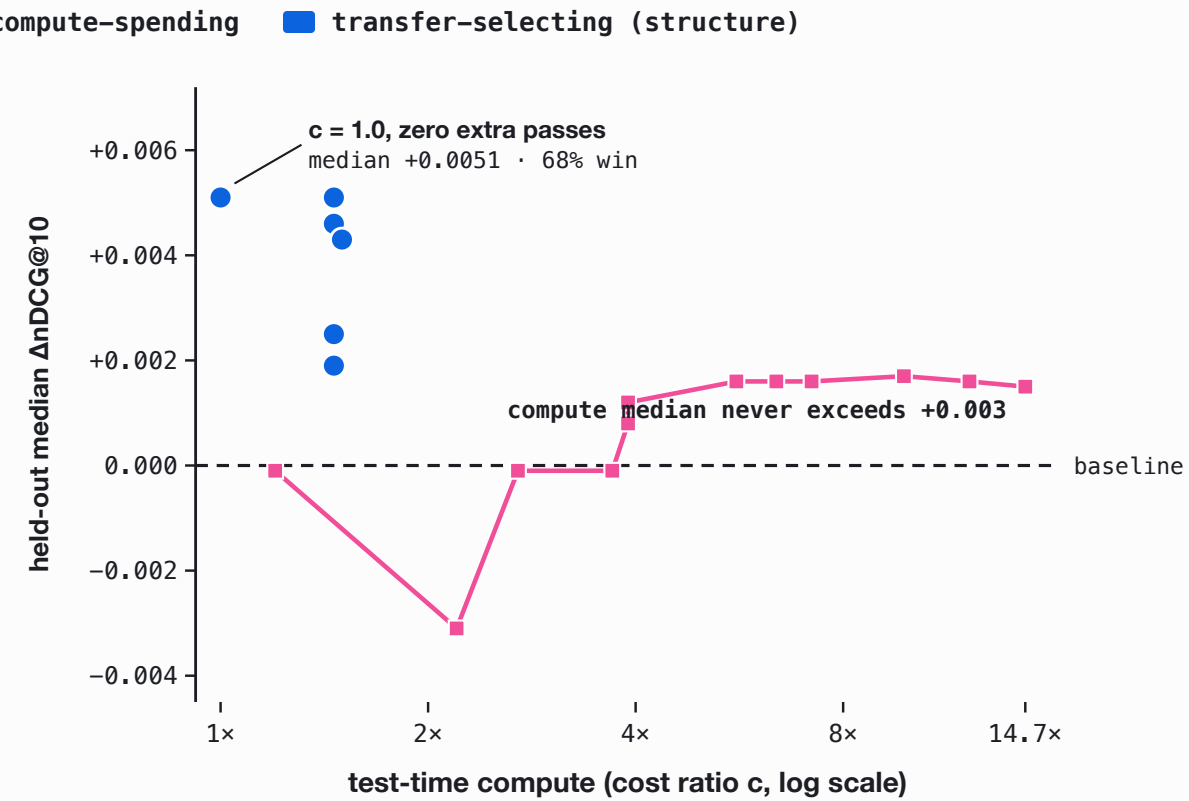


FisherStability c=14.7

Every one is a training-free recombination of the same frozen vectors: chunking, z-scoring, feedback, fusion. Cost climbs left to right; the gains, as the next slide shows, do not.

WHAT IT BUYS ON HELD-OUT DATA

On unseen encoders, compute is flat. Cheap structure is not.



14.7×

the most compute a program spends, beaten flat by a zero-pass program at $c = 1.0$

−0.016

compute's pooled held-out mean, below baseline

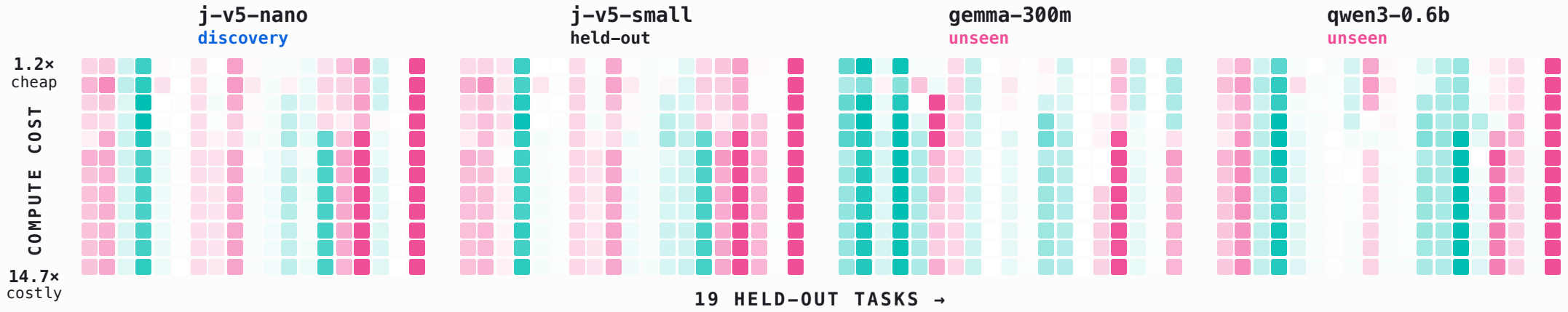
−0.98

its worst per-query cell

The transfer-selecting program at $c = 1.0$, zero extra forward passes, already beats the most expensive compute program at $c = 14.7$. Median is plotted, robust to the -0.98 tail; the pooled mean is negative too.

THE COMPUTE FRONTIER, CELL BY CELL

Compute helps about half the cells, and collapses the rest.



−0.98 +0.16

Each row is one of twelve programs ordered by compute cost (1.2× to 14.7×); each column is one of nineteen held-out tasks; four encoders. 52 of 76 model-task pairs admit some improvement, yet the deep-pink cells fall to −0.98, so the pooled mean stays negative at −0.016.

THE STRUCTURE THAT SURVIVES

Transferable programs cost at most 1.5×. The best adds zero encoder calls.

PROGRAM	c	MEDIAN	WIN-RATE	MEAN	WORST CELL
ZeroCost-Consensus	1.00	+0.0051	68.4%	+0.0148	−0.1070
Deca-Axis	1.46	+0.0051	68.4%	+0.0133	−0.0365
Dodeca-v077	1.46	+0.0046	66.7%	+0.0126	−0.0420
Transpose-Consensus	1.50	+0.0043	83.3%	+0.0219	0.0000
Cross-Axis	1.46	+0.0025	64.9%	+0.0097	−0.0240
Penta-Axis	1.46	+0.0019	59.6%	+0.0107	−0.0485

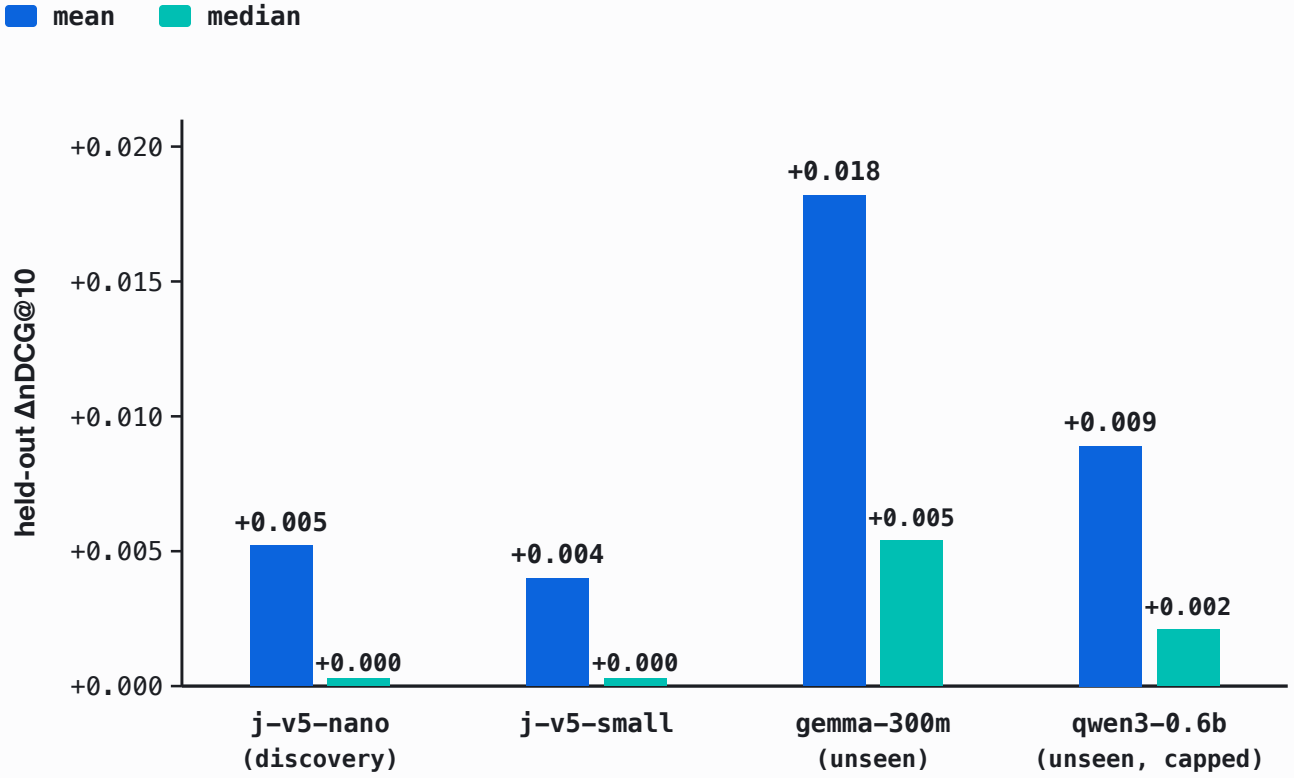
0

losing cells: Transpose-Consensus, at an 83% held-out win-rate

Held-out quality is negatively correlated with cost ($r = -0.37$). The worst cell over the whole table is −0.107: nine times smaller than the compute frontier's −0.98 collapse. The transfer frontier turns the held-out mean positive by removing the catastrophic tail, not by relying on a higher win-rate.

IT TRANSFERS

Gains are largest on encoder families never seen in discovery.



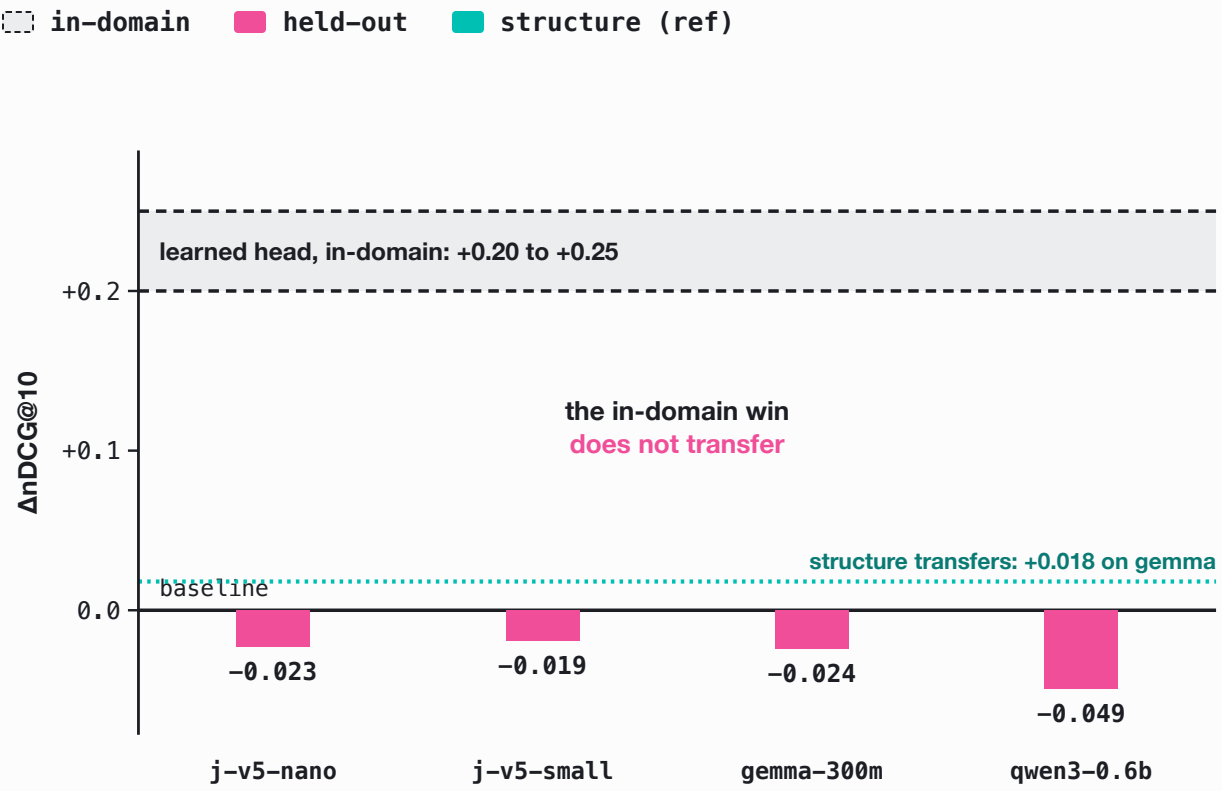
Discovered on `j-v5-nano`. Mean ΔnDCG@10 is positive on all four families, and largest on the two never seen during discovery. The transfer follows general embedding geometry, not artifacts of the discovery encoder.

ACROSS LANGUAGES, TOO

Applied unmodified to French and Greek: median **+0.016**, an **86%** win-rate, and every held-out cell positive on `gemma-300m`.

WHY NOT JUST TRAIN A HEAD?

A matched-budget learned head memorizes. Structure transfers.



Trained on the same 14 discovery tasks, a linear, low-rank, or MLP head improves in-domain retrieval by **+0.20 to +0.25** ΔnDCG@10, yet falls below baseline on **every** held-out encoder.

Adding parameters at the same data budget does not transfer. Recombining the frozen geometry does.

WHAT THE SEARCH FOUND

The transferable programs are classical IR, re-derived in embedding space.

Reciprocal Rank Fusion
REDISCOVERED, NOT SEEDED

Fisher Linear Discriminant
REDISCOVERED, NOT SEEDED

Rocchio Pseudo-Relevance Feedback
OPERATIONALIZED FROM A SEEDED IDEA

Sentence-level MaxSim
OPERATIONALIZED FROM A SEEDED IDEA

The reusable structure is geometric and old: z-scoring, sub-document granularity, centroid feedback. It depends on cosine geometry, not on model-specific training, which is why it carries to encoder families never seen during discovery.

RUNNING THE LOOP

The catch: the loop optimizes exactly what you tell it.

V2 OPTIMIZED WHAT WE TOLD IT

fake in-domain mean, plus "spend more compute", gave a fake scaling frontier to $c = 14.7$ that already saturated by $4\times$

−0.98 a **tail-blind** objective, so held-out cells collapse (gemma 0.99 to 0.01)

all 12 held-out mean negative for **every** program: transfer was never in the loss

V3 OPTIMIZES THE CLAIM

split train / a **hidden validation gate** / test touched once: select on validation, report on test

floor validation median with a **worst-case floor**: kills the catastrophic tail at the source

strip **paired-bootstrap** admission, and **drop the compute mandate** from the prompt

---

A loop hill-climbs the metric you wrote, not the one you meant. Anti-patterns the ledger ruled out: pseudo-doc-as-query −0.49, stacking four LoRAs −0.45, first-100-chars truncation −0.43.

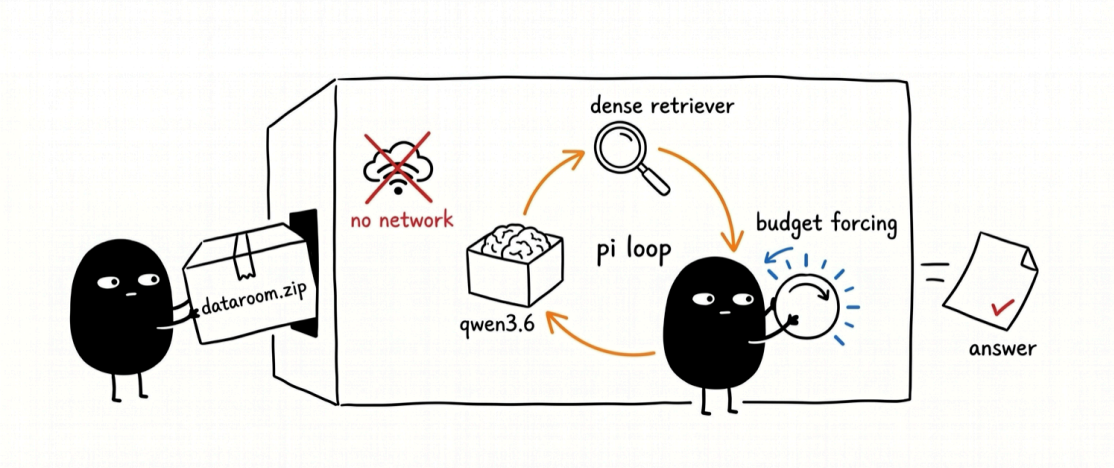
INTERPRETATION B

searchbox: a small agent wires the pipeline itself.

A self-hosted `Qwen3.6-35B-A3B` (~3B active) in an **airgapped** harness explores a `.zip` dataroom under a budget, composing its own pipeline from local tools: grep, embed, rerank, similarity, cluster, select_diverse. No network, near-zero token cost. EXPERIMENTAL

OPEN RESEARCH QUESTIONS

- Which tool does the agent reach for first?
- Is grep all you need: where does a dense retriever add nothing?
- Does forcing more token budget (scaling TTC) help on the hard questions?



the airgapped loop, illustrated · live at hanxiao.io/searchbox

THE UPSTREAM STAGE

dataroom: a small model builds the corpus first.

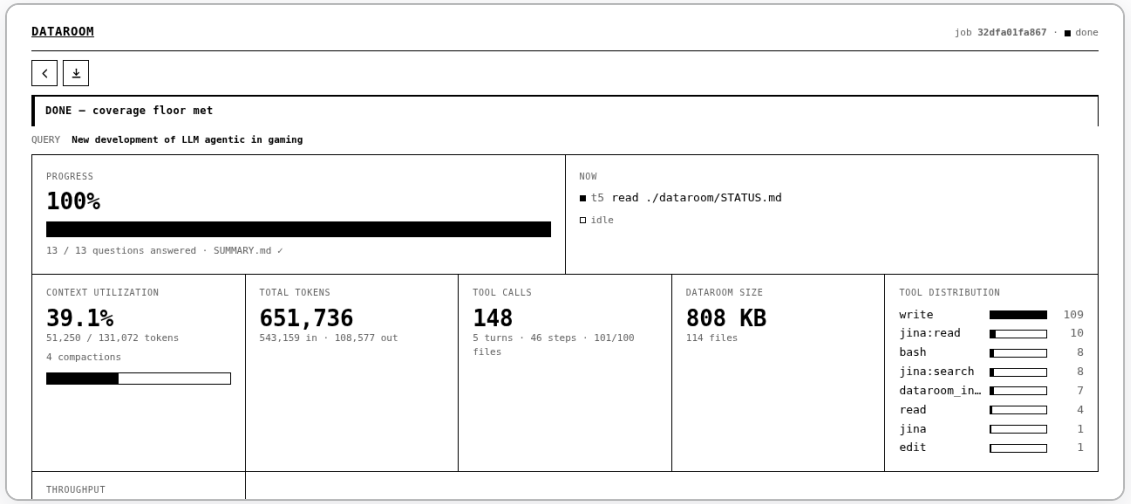
Same harness, one stage earlier. A local model loops **search-read-write** into a fully cited `.zip` corpus, built for a machine to consume rather than a human to skim. Research is mechanical, so it runs on your own GPU at near-zero token cost until the dataroom is comprehensive.

dataroom
`build the corpus (.zip)`

↓

searchbox, or a frontier model
`answer from the .zip`

`Stage one of two: hand the grounded .zip to searchbox or a frontier model for the expensive second stage.`



Live job page: progress to the coverage floor, token usage, and the tool-call distribution. Outcome-based stopping, not a token budget.

CONNECTING THE DOTS

Both manufacture a search pipeline at test time. Neither grows the model.

	A EMBEDDING PROGRAMS	B SEARCHBOX
who assembles it	an agentic search loop, offline	a small agent LLM, at query time
the pipeline is	multi-pass embedding algebra	a chain of retrieval tools
the small model	frozen encoder, 239M	Qwen3.6, ~3B active
what scales	structure, not forward passes	tool composition, not parameters

Test-time compute for small models is pipeline assembly, not raw compute.

THE TAKEAWAY

For frozen retrieval, test-time structure substitutes for model size. Test-time compute does not.

01 Small distilled models have real, transferable test-time potential, realized as structure, not raw compute.

02 Two ways to spend it: recombine the encoder's geometry, or let an agent compose the tools. Both assemble a pipeline at inference.

03 Trust the gain only when the loop optimizes transfer, with a held-out gate it cannot see and a worst-case floor.

~1×

query-time cost. 0.011 s of algebra against 40 s of encoding: the axis that transfers is also the cheapest to serve.

Honest caveat: absolute held-out gains are modest, pooled means of +0.004 to +0.018 ΔnDCG@10. The claim is about the shape of the compute-quality relation and the tail, not magnitude.

AI Engineer World's Fair

THANK YOU

Spend structure, not compute.

Zero training. Across frozen encoders. About 1× serving cost.

Han Xiao · VP of AI, Elastic

github.com/hanxiao / embedding-ttc · searchbox · dataroom

arXiv:2605.11374

@hxiao · in/hxiao87

THESE SLIDES

hanxiao.io/ttc-aie-sf